Greedy

Greedy

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.

A **greedy algorithm** always makes the choice that looks best at the moment.

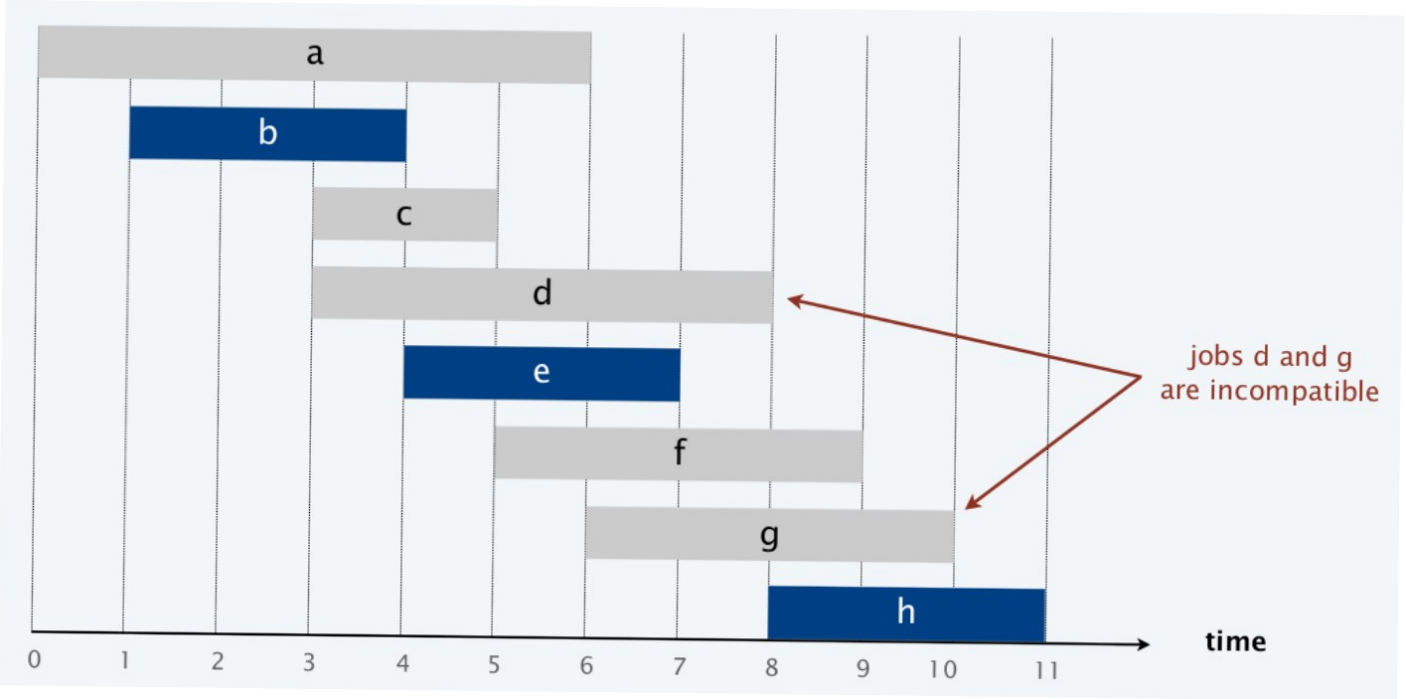
That is, it makes a **locally optimal choice** in the hope that this choice will lead to a **globally optimal solution**.

Question: Does greedy algorithm always yield optimal solution?

Interval Scheduling

Input: A set of jobs with start and finish times,

Question: find the maximum subset of mutually compatible (non-overlapping) jobs.



Interval Scheduling

EARLIEST-FINISH-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT jobs by finish times and renumber so that $f_1 \leq f_2 \leq \ldots \leq f_n$.

$S \leftarrow \emptyset$. ← set of jobs selected

FOR $j = 1$ TO n

  IF (job j is compatible with S)

    $S \leftarrow S \cup \{ j \}$.

RETURN S.

Interval Scheduling

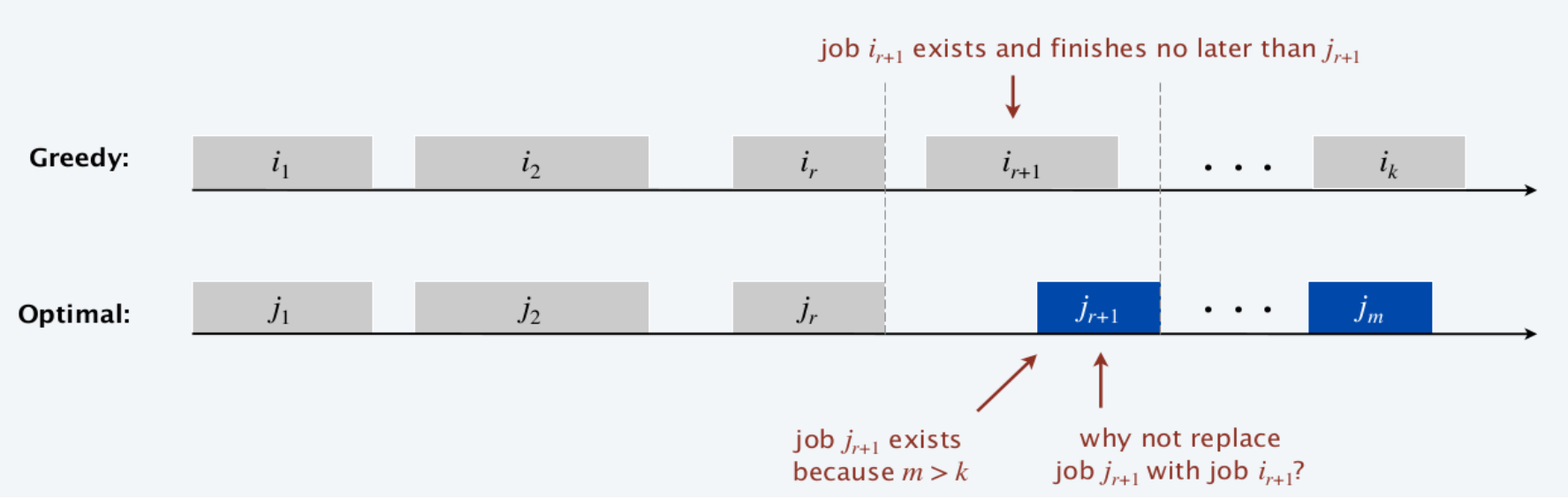
Time: O(n log n) time.

1. Keep track of the job j^* that was added last to S
2. Job j is compatible with S if and only if $s_j \geq f_{j^*}$
3. Sorting by finish times takes O(n log n) time.

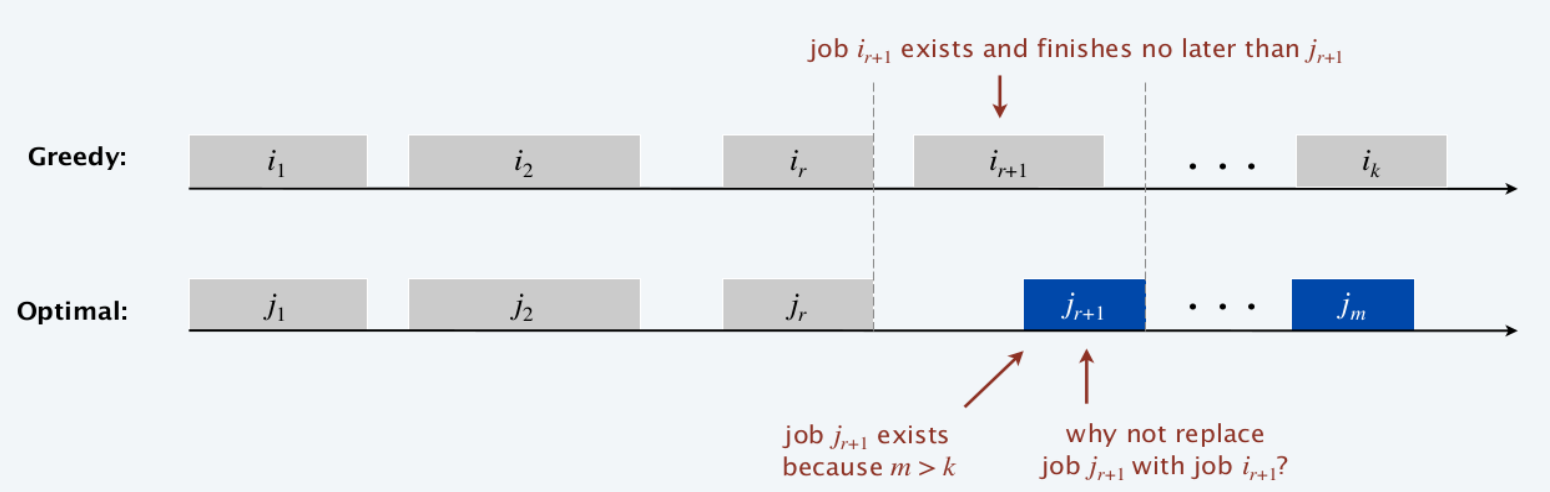
Interval Scheduling

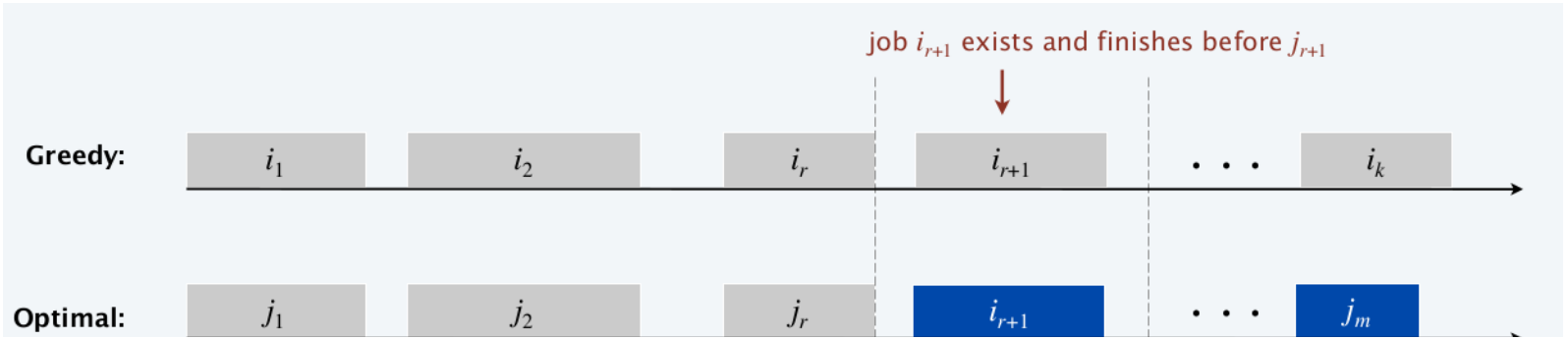
Correctness of Algorithm:

Suppose that greedy is not optimal. What happens?



Interval Scheduling





Interval Scheduling

Question: Suppose that each job also has a positive weight and the goal is to find a maximum weight subset of mutually compatible intervals. Is the **earliest-finish-time-first** algorithm still **optimal**?

A. Yes, because greedy algorithms are always optimal.

B. Yes, because the same proof of correctness is valid.

C. No, because the same proof of correctness is no longer valid.

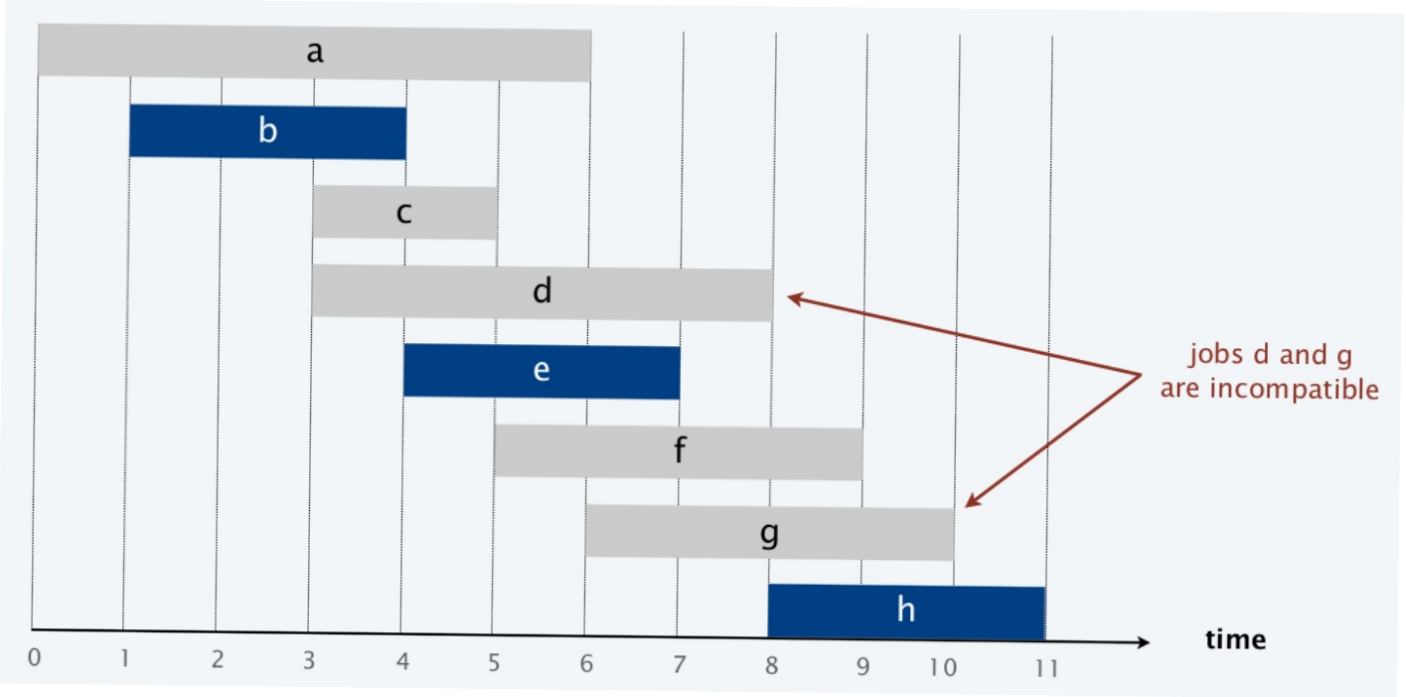
D. No, because you could assign a huge weight to a job that overlaps the job with the earliest finish time.

Interval Partitioning

Interval Partitioning

Input: A set of lectures with start and finish times,

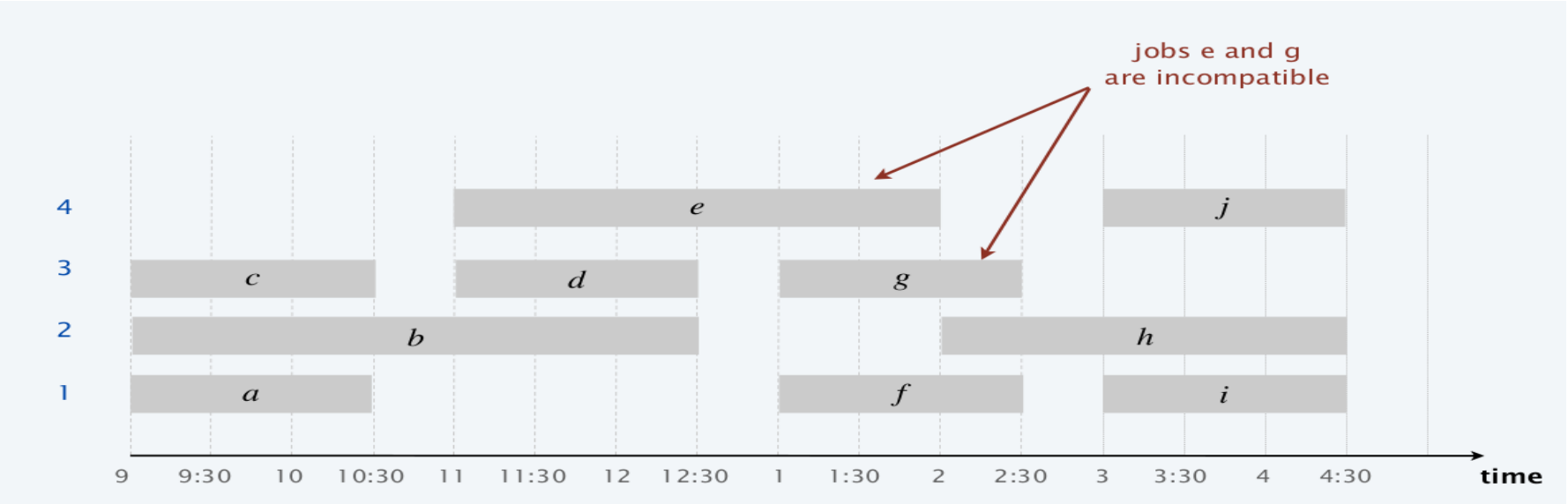
Question: find the minimum number of classrooms to schedule all the lectures so that no two lectures happen at the same time in the same room.



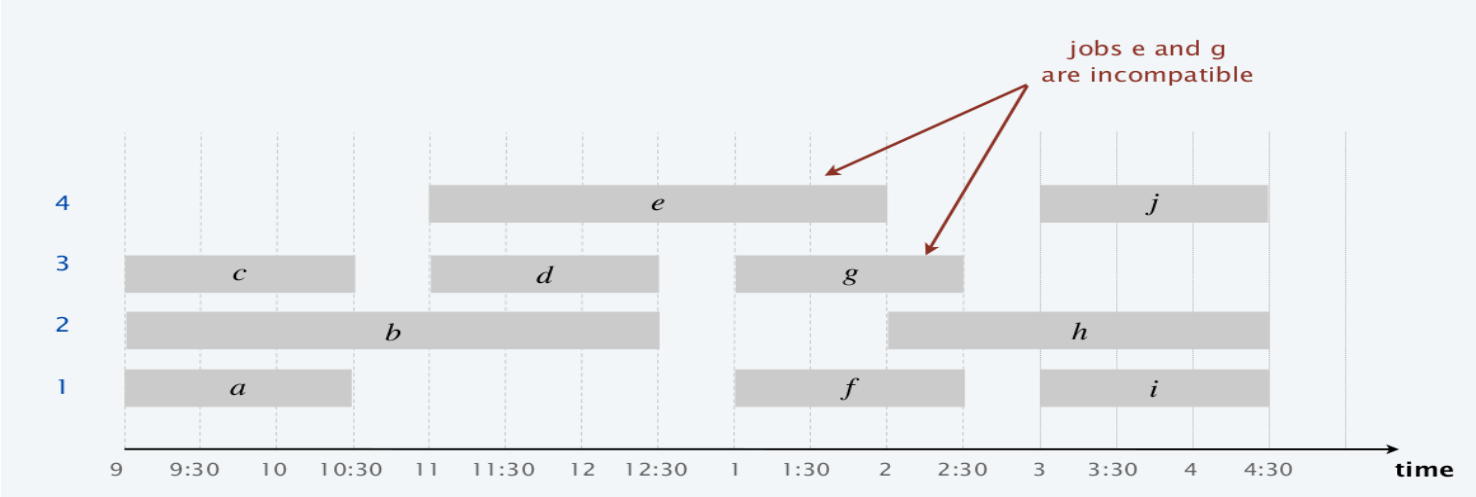
Interval Partitioning

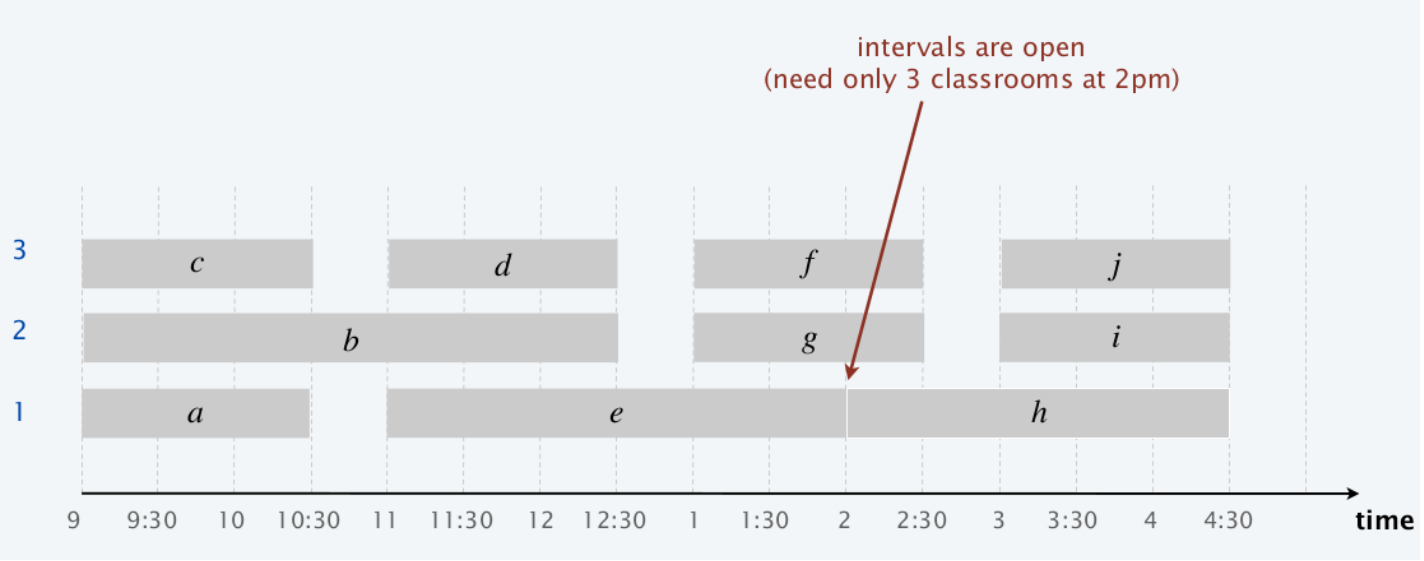
Input: A set of lectures with start and finish times,

Question: find the minimum number of classrooms to schedule all the lectures so that no two lectures happen at the same time in the same room.



Interval Partitioning





Question

Consider lectures in some order, assigning each lecture to first available classroom (opening a new classroom if none is available). Which rule is optimal?

A. [Earliest start time] Consider lectures in ascending order of s_j.

B. [Earliest finish time] Consider lectures in ascending order of f_j.

C. [Shortest interval] Consider lectures in ascending order of $f_j - s_j$.

D. None of the above.

Earliest_Start_Time_First

Earliest_Start_Time_First

EARLIEST-START-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT lectures by start times and renumber so that $s_1 \leq s_2 \leq \ldots \leq s_n$.

$d \leftarrow 0$. ← number of allocated classrooms

FOR $j = 1$ TO n

  IF (lecture j is compatible with some classroom)

    Schedule lecture j in any such classroom k.

  ELSE

Earliest_Start_Time_First

EARLIEST-START-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT lectures by start times and renumber so that $s_1 \leq s_2 \leq \ldots \leq s_n$.

$d \leftarrow 0$. ← number of allocated classrooms

FOR $j = 1$ TO n

  IF (lecture j is compatible with some classroom)

    Schedule lecture j in any such classroom k.

  ELSE

    Allocate a new classroom $d + 1$.

    Schedule lecture j in classroom $d + 1$.

    $d \leftarrow d + 1$.

RETURN schedule.

Analysis

- What happens if there are multiple classrooms compatible with a lecture under consideration?

Analysis

- What happens if there are multiple classrooms compatible with a lecture under consideration?
    - This implementation chooses a classroom k whose finish time of its last lecture is the earliest.

Analysis

EARLIEST-START-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT lectures by start times and renumber so that $s_1 \leq s_2 \leq \ldots \leq s_n$.

$d \leftarrow 0$. ← number of allocated classrooms

FOR $j = 1$ TO n

  IF (lecture j is compatible with some classroom)

    Schedule lecture j in any such classroom k.

  ELSE

    Allocate a new classroom $d + 1$.

    Schedule lecture j in classroom $d + 1$.

    $d \leftarrow d + 1$.

RETURN schedule.

Analysis

- Sorting by **start times** takes O(n log n) time.
- Store classrooms in a **priority queue** (key = finish time of its last lecture).
  - to allocate .
  - to schedule .
  - to determine whether lecture j is compatible with any classroom,

- Total # of priority queue operations is

Analysis

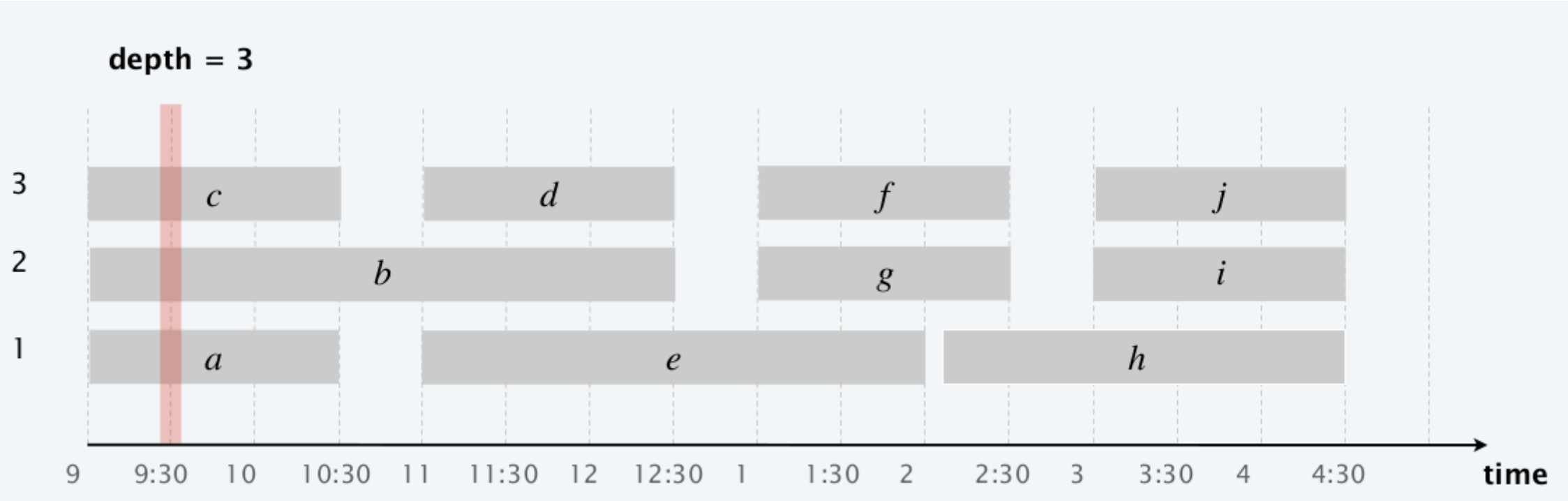
- Sorting by **start times** takes O(n log n) time.
- Store classrooms in a **priority queue** (key = finish time of its last lecture).
  - to allocate a new classroom, INSERT classroom onto priority queue.
  - to schedule lecture j in classroom k, INCREASE-KEY of classroom k to f_j.
  - to determine whether lecture j is compatible with any classroom, compare s_j to FIND-MIN
- Total # of priority queue operations is

Analysis

- Sorting by **start times** takes O(n log n) time.
- Store classrooms in a **priority queue** (key = finish time of its last lecture).
  - to allocate a new classroom, INSERT classroom onto priority queue.
  - to schedule lecture j in classroom k, INCREASE-KEY of classroom k to f_j.
  - to determine whether lecture j is compatible with any classroom, compare s_j to FIND-MIN
- Total # of priority queue operations is O(n); each takes O(log n) time

Correctness

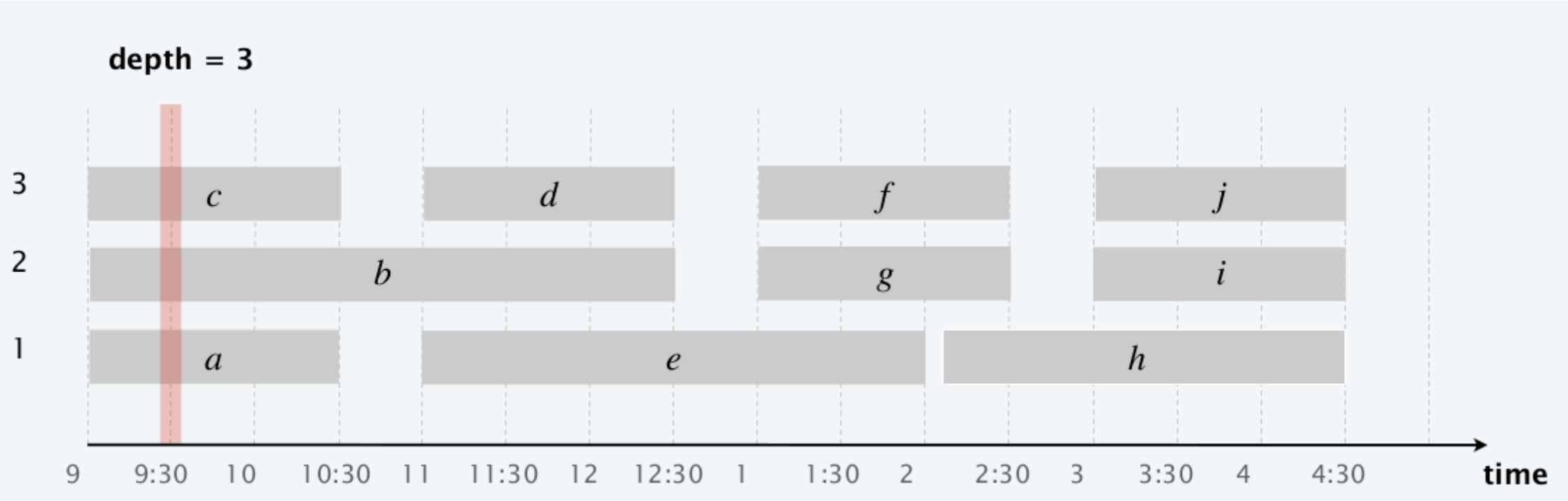
- The **depth** of a set of open intervals is the maximum number of intervals that contain at any given point.



Correctness

- The **depth** of a set of open intervals is the maximum number of intervals that contain at any given point.

Number of classrooms **required** ≥ depth.



Correctness

- **Observation.** The earliest-start-time first algorithm never schedules two incompatible lectures in the same classroom.

Correctness

- d → number of classrooms that the algorithm allocates.

Correctness

- d → number of classrooms that the algorithm allocates.

Why is the classroom C_d open?

Correctness

- d → number of classrooms that the algorithm allocates.
- Classroom d is opened **because** we needed to schedule a lecture, say j, that is incompatible with a lecture in each of d – 1 other classrooms.

Correctness

- d → number of classrooms that the algorithm allocates.
- Classroom d is opened **because** we needed to schedule a lecture, say j, that is incompatible with a lecture in each of d – 1 other classrooms.

Thus, each of these d lectures end after s_j.

Correctness

- d → number of classrooms that the algorithm allocates.
- Classroom d is opened **because** we needed to schedule a lecture, say j, that is incompatible with a lecture in each of d – 1 other classrooms.

Thus, each of these d lectures end after s_j.

- Since we sorted by start time, each of these incompatible lectures start no later than s_j.

Correctness

- d → number of classrooms that the algorithm allocates.
- Classroom d is opened **because** we needed to schedule a lecture, say j, that is incompatible with a lecture in each of d – 1 other classrooms.

Thus, each of these d lectures end after s_j.

- Since we sorted by start time, each of these incompatible lectures start no later than s_j.

We have d lectures overlapping at time $s_j + \varepsilon$.

Correctness

- Any schedule uses ≥ d classrooms (Lower bound)

- Since we had a strategy to schedule using d classes,
    - optimal #classrooms ≤ d (Upper bound)